

Noida Institute of Engineering and Technology
(Autonomous)

Regulation 2025-26, Greater Noida



B-Tech
in
CSE-Artificial Intelligence & Machine Learning
(3rd Year, VI Sem., Session: 2025-26)

LAB MANUAL

Software Engineering Lab (ACSE0653)
(6th Semester)

Noida Institute of Engineering and Technology
19, Institutional Area, Knowledge Park II, Greater Noida
Uttar Pradesh – 201306, <http://www.niet.co.in>

SOFTWARE ENGINEERING LAB(ACSE0653)
INDEX

S.No.	Practical	Date	Sign
1	Team formation and allotment of Mini project: Problem statement, Literature survey, Requirement analysis.		
2	Draw the use case diagram: specify the role of each of the actors, Data Flow Diagram (DFD): All levels.		
3	Design an ER diagram with multiplicity.		
4	Prepare an SRS document in line with the IEEE recommended standards.		
5	Create a Software Design Document (SDD): Object and Class diagram.		
6	Create Interaction diagram: sequence diagram, collaboration diagram for SDD.		
7	Create Activity diagram and Component diagram for SDD.		
8	Estimation of Test Coverage Metrics and Structural Complexity.		
9	Design test suite for equivalence class partitioning.		
10	Design test cases for Boundary value analysis.		

Experiment 1

OBJECTIVE: Team formation and allotment of Mini project: Problem statement, Literature survey, Requirement analysis.

Team Member(s):

Piyush Jain (2301331530112)

Ramandeep singh (2301331530132)

Kartik Garg (2301331530083)

Prabhat Dubey (2301331530117)

Topic of Mini Project -

Lab Management System for an Educational Institute

Problem Statement -

Manual management of laboratories in educational institutes involves maintaining lab schedules, equipment records, and user details using paper-based methods. This approach is time-consuming, error-prone, and lacks proper data security. There is a need for an automated Lab Management System that can efficiently manage laboratories, equipment, and lab usage while providing secure access to authorized users.

Literature Survey -

- Studied existing manual laboratory record management systems.
- Reviewed basic laboratory management software used in educational institutes.
- Analyzed limitations such as lack of automation, data redundancy, and security issues.

- Identified the need for a centralized system to manage labs and equipment

Requirement Analysis -

1. User Requirements

- The system shall be secure and accessible only to authorized users.
- The system shall provide reliable and accurate data.
- The system shall respond within acceptable time limits.
- The system shall be user-friendly and easy to use.

2. Functional Requirements

- The system shall provide user login and authentication.
- The system shall allow management of laboratory details.
- The system shall manage laboratory equipment records.
- The system shall support lab allocation and scheduling.
- The system shall generate reports related to lab usage and equipment.

3. Technical Requirements

- Frontend: Next.js, Tailwind CSS, TypeScript
- Backend: Next.js API routes (Node.js environment)
- Database: PostgreSQL (via Xata or any relational DB like MySQL)
- ORM: Drizzle ORM (for database handling)
- Authentication & Authorization: Better Auth (Role-based access: Admin, Faculty, Student)
- Hosting: Vercel / Netlify (Frontend & API deployment)

- Security:
- Arcjet for rate limiting & bot protection
- Data encryption for sensitive information
- Secure login with JWT/session-based auth
- API Integration: RESTful APIs for communication between frontend & backend
- Cross-Platform Compatibility: Works on Desktop, Tablet, and Mobile
- Version Control: Git & GitHub for collaboration

4. Usability and User Experience

- Simple and clean user interface using Tailwind CSS
- Easy navigation dashboard for Admin, Faculty, and Students
- Responsive design for all devices (mobile, tablet, desktop)
- Quick access to key features like lab booking, equipment status, and reports
- Search and filter options for labs and equipment
- Real-time updates for lab availability and schedules
- Error handling and notifications (e.g., booking conflicts, login errors)
- Consistent UI components for better user experience
- Fast loading time for smooth performance

Experiment 2

OBJECTIVE: Draw the use case diagram: specify the role of each of the actors, Data Flow Diagram (DFD): All levels

2.1 Purpose of Use Case Diagram

The Use Case Diagram represents the interaction between users and the Lab Management System. It identifies different users (actors) and the services (use cases) provided by the system.

2.2 Actors Identified

The following actors interact with the Lab Management System:

- **Admin**
- **Lab Assistant**
- **Faculty**
- **Student**

2.3 Use Cases Identified

The Lab Management System provides the following use cases:

- Login
- Manage Labs
- Manage Equipment
- Allocate Lab
- Issue/Return Equipment
- View Lab Schedule

2.4 Actor–Use Case Interaction

- Admin can login, manage labs, manage equipment, allocate labs, and view reports.

- Lab Assistant can login and issue or return equipment.
- Faculty can login and view lab schedules.
- Student can login and view lab availability and schedules.

2.5 Rules Followed While Drawing the Use Case Diagram

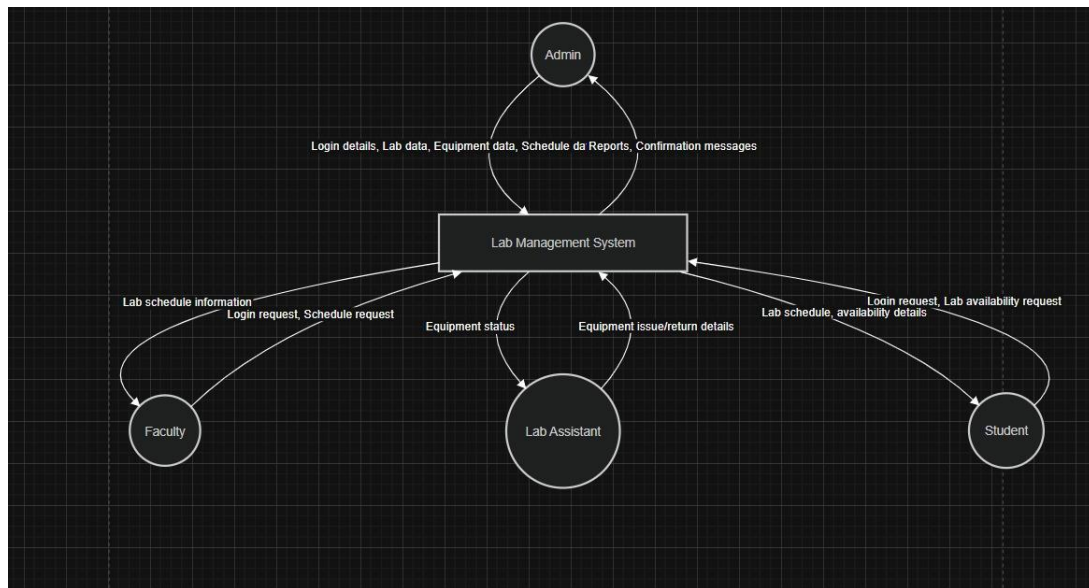
- System boundary is named “**Lab Management System**”
- Actors are placed outside the system boundary
- Use cases are placed inside the system boundary
- Only association lines are used
- No include or extend relationships are used

2.6 Tool Used

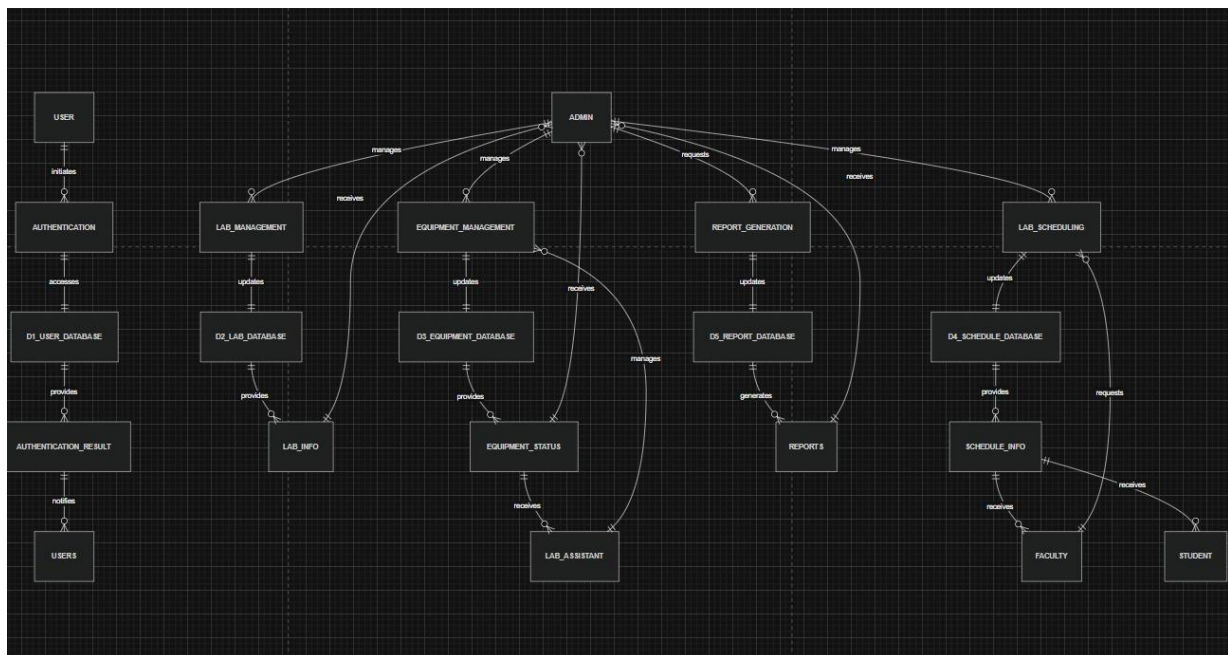
The Use Case Diagram is created using **draw.io** (**diagrams.net**).

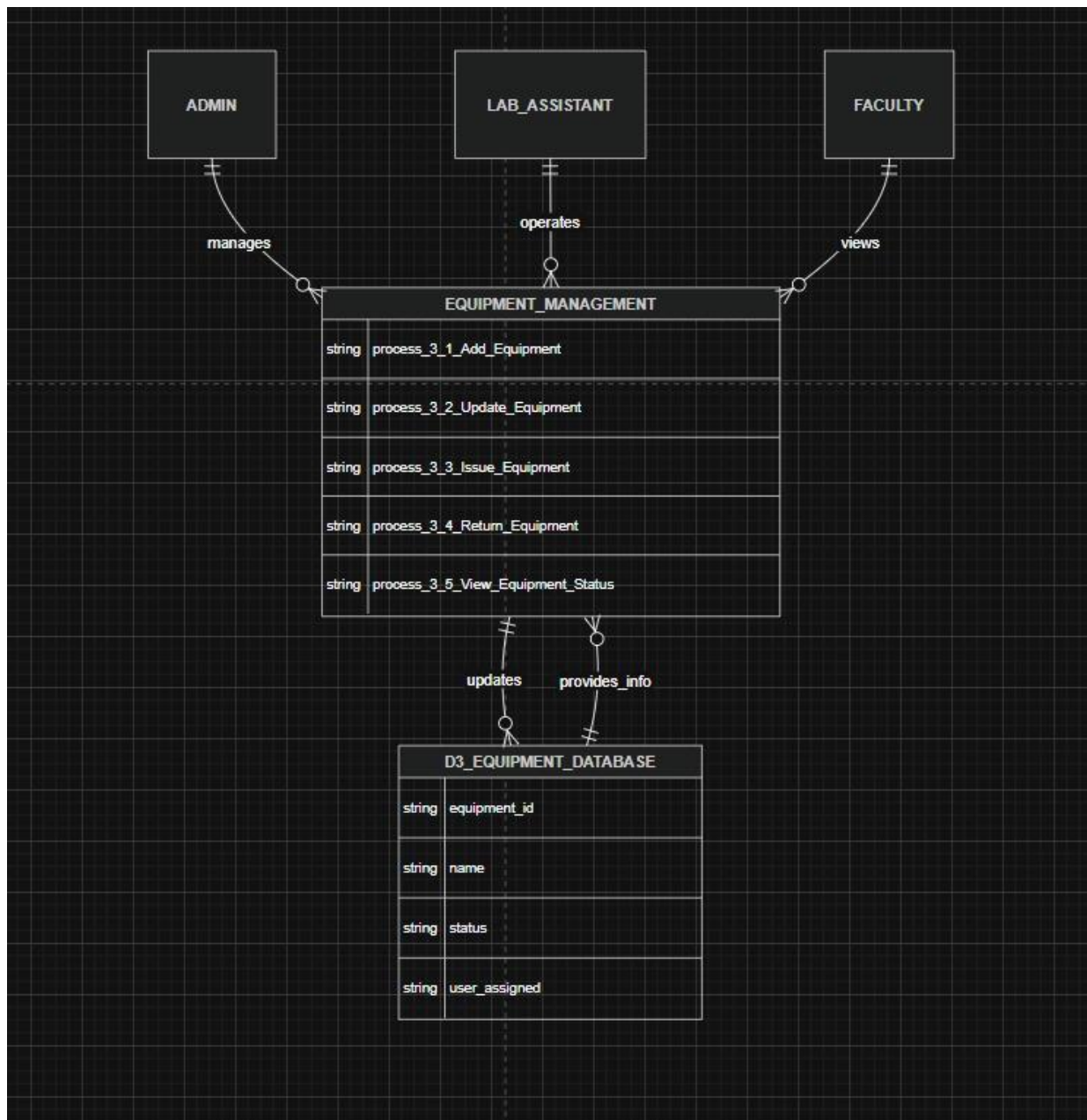
DIAGRAM:

Level 0 DFD :



Level 1 DFD:



Level 1 DFD:

Experiment 3

OBJECTIVE: Design an ER diagram for with multiplicity.

1. Entities & Attributes

ADMIN

- Admin_ID (PK)
- Name
- Email

LAB

- Lab_ID (PK)
- Lab_Name
- Location
- Capacity

EQUIPMENT

- Equipment_ID (PK)
- Name
- Quantity
- Status

FACULTY

- Faculty_ID (PK)
- Name
- Department

SCHEDULE

- Schedule_ID (PK)
- Date
- Time_Slot

STUDENT

- Student_ID (PK)
- Name
- Branch

ATTENDANCE

- Attendance_ID (PK)
- Status
- Date

REPORT

- Report_ID (PK)
- Report_Type
- Generated_Date

2. Relationships

- ADMIN → LAB
One Admin manages many Labs (1 : M)
- LAB → EQUIPMENT
One Lab contains many Equipment (1 : M)
- LAB → SCHEDULE
One Lab is assigned many Schedules (1 : M)
- FACULTY → SCHEDULE
One Faculty creates many Schedules (1 : M)
- SCHEDULE → ATTENDANCE
One Schedule records many Attendance entries (1 : M)
- STUDENT → ATTENDANCE
One Student has many Attendance records (1 : M)
- LAB / SYSTEM → REPORT
System generates Reports (1 : M relationship implied)

3. Attributes Explanation

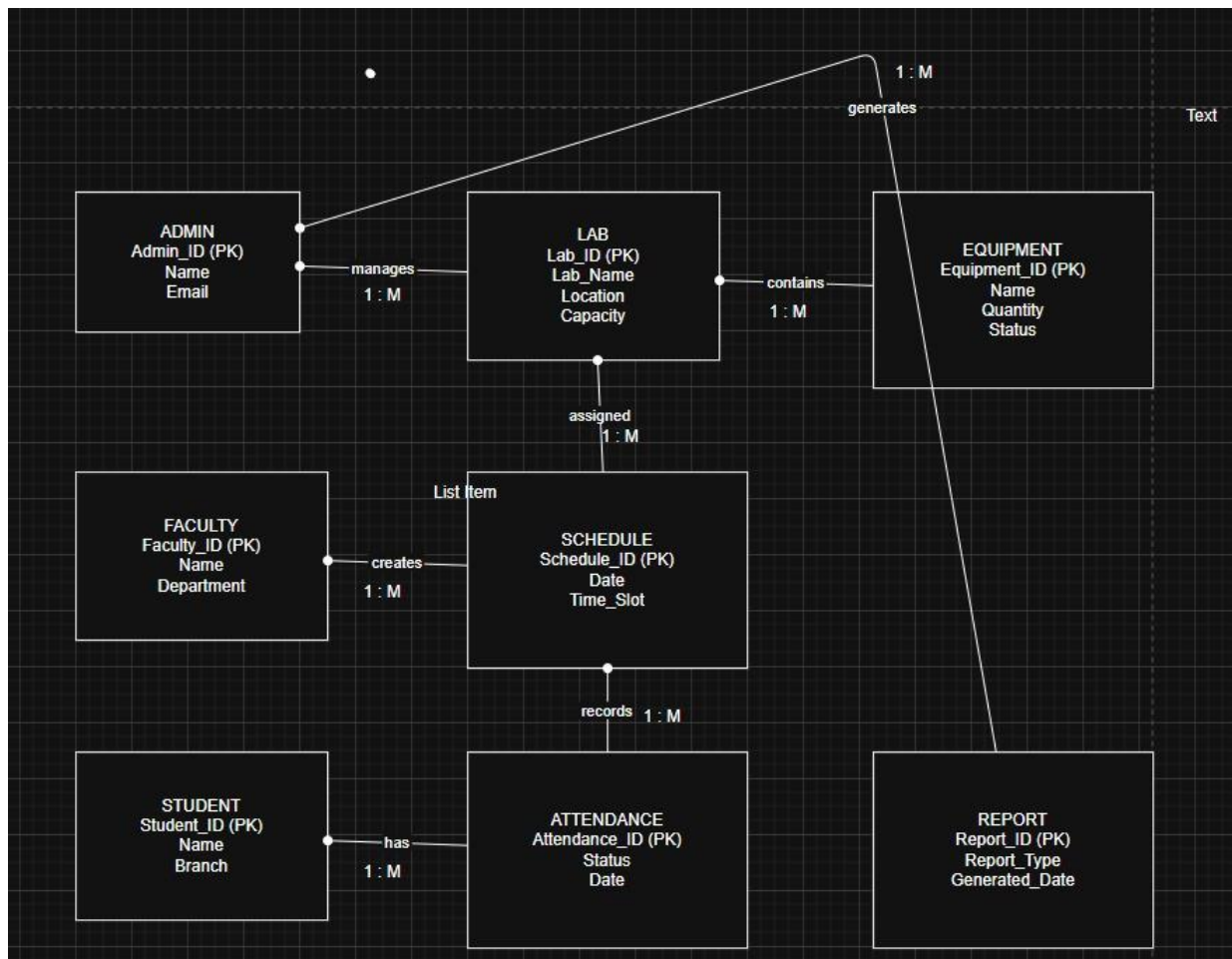
Each entity contains attributes that describe its properties:

- Admin → stores administrator details
- Lab → stores lab information like location & capacity
- Equipment → tracks lab resources and their status
- Faculty → manages lab schedules
- Schedule → defines lab usage (date & time)
- Student → stores student details
- Attendance → records student presence

- Report → stores generated reports for analysis

These attributes help in:

- Efficient data storage
- Easy retrieval of information
- Maintaining relationships between entities



Experiment 4

OBJECTIVE: Prepare a SRS document in line with the IEEE recommended standards.

Software Requirements Specification(SRS) Document

Lab Management System for Educational Institute

1. Introduction

• Purpose

This document provides a detailed Software Requirements Specification (SRS) for the Lab Management System. It defines system functionality, features, constraints, and requirements to guide development and ensure clear communication among stakeholders.

• Scope

The system is designed to automate laboratory management in educational institutes. It allows administrators to manage labs, equipment, schedules, and users efficiently. Faculty can schedule lab sessions, and student attendance can be recorded digitally.

• Definitions, Acronyms, and Abbreviations

- SRS – Software Requirements Specification
- UI – User Interface
- API – Application Programming Interface
- DBMS – Database Management System
- ORM – Object Relational Mapper

2. Overall Description

- **Product Perspective**

This is a web-based application that replaces manual lab management systems. It uses modern technologies like Next.js and PostgreSQL to ensure efficiency, scalability, and security.

- **User Classes and Characteristics**

- **Admin**

- Manages labs, equipment, and users
- Generates reports

- **Faculty**

- Creates lab schedules
- Manages student attendance

- **Student**

- Views schedules
- Attendance is recorded

- **Operating Environment**

- Frontend: Web browsers (Chrome, Edge, Firefox, Safari)
- Backend: Node.js (Next.js API routes)
- Database: PostgreSQL (Xata or similar)
- Devices Supported: Desktop, Tablet, Mobile

- **Design and Implementation Constraints**

- Must use TypeScript
- Responsive design required
- Secure authentication system
- System should handle multiple users simultaneously

3. System Features

- User login and authentication
- Lab management (add, update, delete labs)
- Equipment management
- Lab scheduling system
- Attendance management
- Report generation
- Role-based access (Admin, Faculty, Student)

4. External Interface Requirements

• User Interface

- Clean and responsive UI using Tailwind CSS
- Dashboard for Admin and Faculty
- Easy navigation menus
- Forms for lab, equipment, and scheduling

• APIs

- /api/auth → Authentication
- /api/lab → Lab management
- /api/equipment → Equipment management
- /api/schedule → Scheduling
- /api/attendance → Attendance
- /api/report → Reports

5. System Requirements

• Functional Requirements

- Users must log in securely
- Admin can manage labs and equipment
- Faculty can create schedules
- System must record attendance
- Reports must be generated automatically
- Students can view schedules

- **Non-Functional Requirements**

- Performance: Fast response time (< 2 seconds)
- Security: Secure login and data protection
- Scalability: Support multiple users
- Usability: Simple and easy interface
- Availability: High system uptime

6. System Models

- **Use Case Diagrams**

Will include:

- Login system
- Lab management
- Scheduling
- Attendance tracking
- Report generation

- **Sequence Diagrams**

Will include:

- Login process
- Schedule creation
- Attendance recording
- Report generation

7. Other Requirements

- **Documentation Requirements**

- Installation guide
- User manual
- API documentation

- **Testing Requirements**

- Unit Testing
- Integration Testing

- System Testing
- User Acceptance Testing (UAT)

8. References

- Next.js Documentation
- PostgreSQL Documentation
- Drizzle ORM Docs
- Tailwind CSS Docs

Experiment 5

OBJECTIVE: Create a Software Design Document(SDD): Object and Class diagram.

1. Introduction

The **Lab Management System** is a web-based application designed to automate the management of laboratories in an educational institute. It handles lab scheduling, equipment tracking, attendance recording, and report generation.

This document defines the **class structure and object-oriented design**, helping developers understand system components and relationships.

2. Objectives

- Define system design using **object-oriented principles**
- Represent system components using **class diagrams**
- Describe relationships between classes
- Ensure **modular and scalable development**

3. Classes

1. Admin

Attributes:

- adminId
- name
- email

Methods:

- login()
- manageLab()

- manageEquipment()
- generateReport()

2. Lab

Attributes:

- labId
- labName
- location
- capacity

Methods:

- addLab()
- updateLab()
- deleteLab()
- assignSchedule()

3. Equipment

Attributes:

- equipmentId
- name
- quantity
- status

Methods:

- addEquipment()
- updateEquipment()
- deleteEquipment()
- checkAvailability()

4. Faculty

Attributes:

- facultyId
- name
- department

Methods:

- createSchedule()
- updateSchedule()
- markAttendance()

5. Schedule**Attributes:**

- scheduleId
- date
- timeSlot

Methods:

- createSchedule()
- updateSchedule()
- deleteSchedule()

6. Student**Attributes:**

- studentId
- name
- branch

Methods:

- viewSchedule()
- viewAttendance()

7. Attendance

Attributes:

- attendanceId
- status
- date

Methods:

- markAttendance()
- updateAttendance()
- viewAttendance()

8. Report

Attributes:

- reportId
- reportType
- generatedDate

Methods:

- generateReport()
- exportReport()

4. Relationships Between Classes

- **Admin** → **Lab** (1:M) → Admin manages multiple labs
- **Lab** → **Equipment** (1:M) → One lab contains many equipment
- **Lab** → **Schedule** (1:M) → Lab has multiple schedules
- **Faculty** → **Schedule** (1:M) → Faculty creates schedules
- **Schedule** → **Attendance** (1:M) → Each schedule records attendance

- **Student** → **Attendance** (1:M) → Student has multiple attendance records
- **System** → **Report** → Reports generated from system data

5. Objects (Instance Level Examples)

Admin Object

```
{ adminId: 1, name: "Admin1", email: "admin@college.com" }
```

Lab Object

```
{ labId: 101, labName: "Computer Lab", location: "Block A",  
capacity: 60 }
```

Equipment Object

```
{ equipmentId: 201, name: "Computer", quantity: 50, status:  
"Available" }
```

Faculty Object

```
{ facultyId: 301, name: "Dr. Sharma", department: "CSE" }
```

Schedule Object

```
{ scheduleId: 401, date: "2026-03-25", timeSlot: "10:00 AM -  
12:00 PM" }
```

Student Object

```
{ studentId: 501, name: "Rahul", branch: "CSE" }
```

Attendance Object

```
{ attendanceId: 601, status: "Present", date: "2026-03-25" }
```

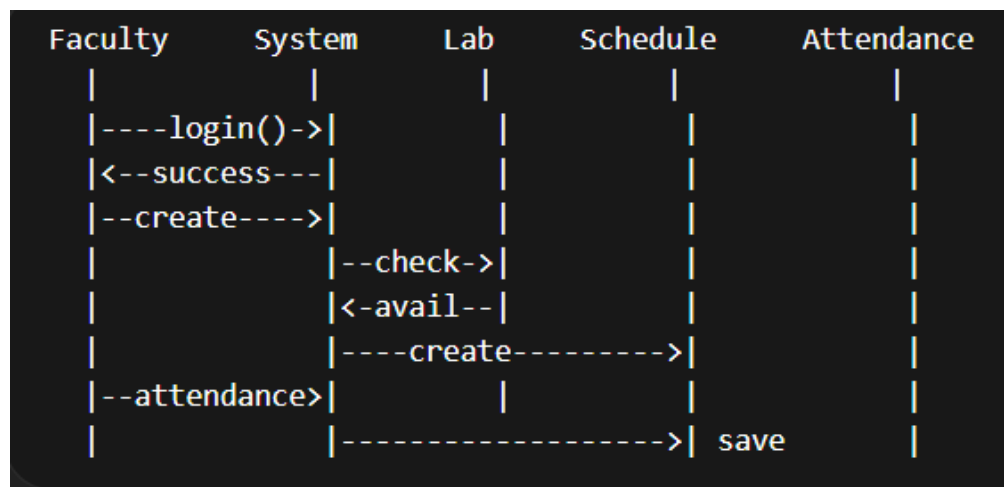
Report Object

```
{ reportId: 701, reportType: "Lab Usage", generatedDate:  
"2026-03-25" }
```

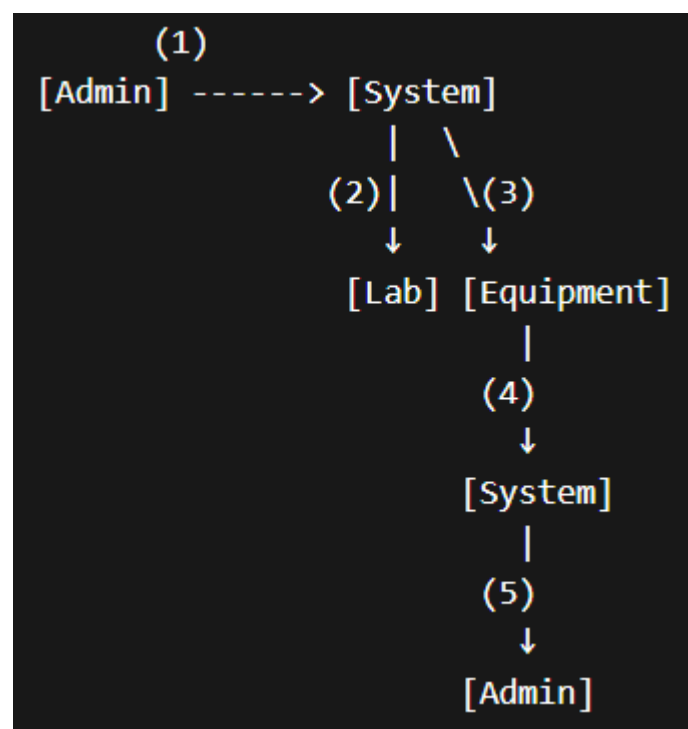
Experiment 6

OBJECTIVE: Create Interaction diagram: sequence diagram, collaboration diagram for SDD.

Sequence Diagram



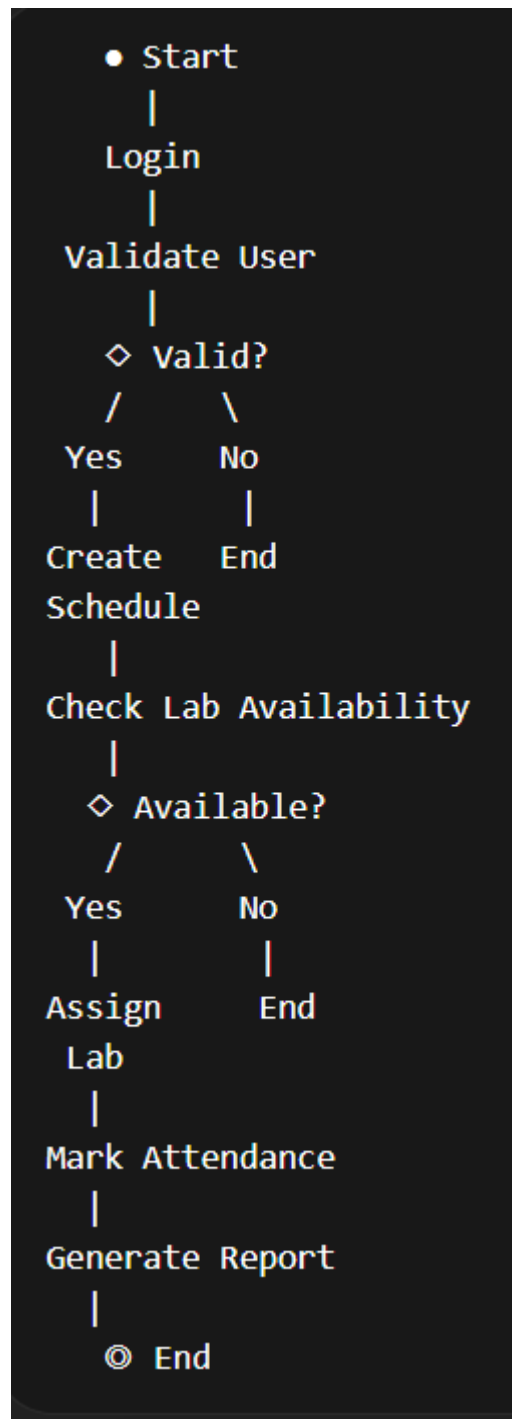
Collaboration Diagram

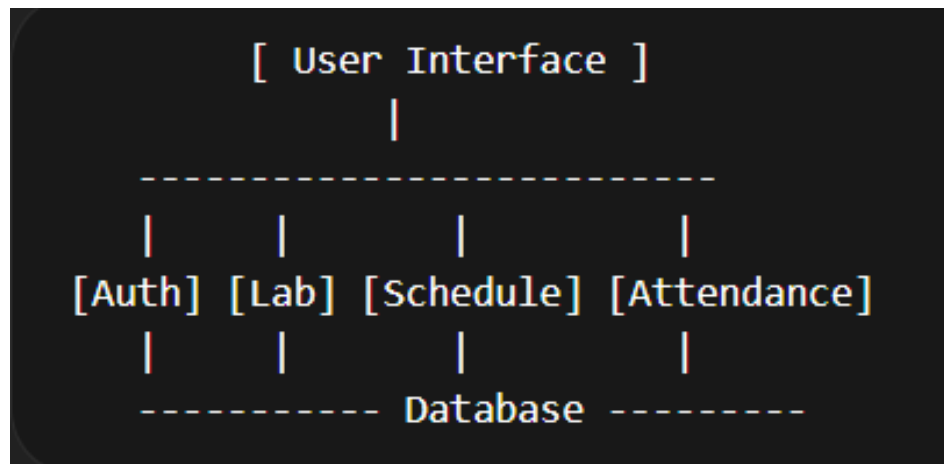


Experiment 7

OBJECTIVE: Create Activity diagram and Component diagram for SDD

Activity Diagram:



Component Diagram:

Experiment 8

Objective: Estimation of Test Coverage Metrics and Structural Complexity

1. Estimation of Test Coverage Metrics

Test coverage metrics are used to evaluate how much of the system code is tested. The following modules are considered:

Modules Covered

- User Authentication
- Lab Management
- Equipment Management
- Scheduling System
- Attendance Management

Coverage Metrics Table

Metric	Description	Example (Lab Management System)
Statement Coverage	% of executed code statements during tests	e.g., 88% of lab module code executed
Branch Coverage	% of decision branches (if/else) tested	e.g., 80% of login validation cases covered
Function Coverage	% of functions executed at least once	e.g., 92% of scheduling functions tested
Condition Coverage	Boolean conditions tested for both true/false	e.g., equipment availability (Available/Not Available)

2. Structural Complexity Estimation

Cyclomatic Complexity is used to measure the complexity of the

system.

Formula

$$V(G) = E - N + 2P$$

Where:

- E = Number of edges
- N = Number of nodes
- P = Number of connected components

Example: Scheduling Module

The scheduling module includes:

- Lab availability check
- Time conflict handling
- Schedule creation

Assume:

- E = 20
- N = 15
- P = 1

Cyclomatic Complexity:

$$\begin{aligned} &[\\ V(G) &= 20 - 15 + 2 = 7 \\ &] \end{aligned}$$

Interpretation

- **Complexity = 7 (Moderate)**
- **Ideal value should be < 10 for maintainability**

3. Tool Recommendations

Tool	Purpose	Used For
Istanbul (nyc)	Code coverage	JavaScript/TypeScript modules
SonarQube	Quality + Complexity	Full system analysis
ESLint + Plugin	Code quality	Detect complexity issues
Jest	Unit testing	API & frontend logic
CodeClimate	Code quality	Maintainability tracking

4. Summary Table

Module	Statement Coverage	Branch Coverage	Cyclomatic Complexity
User Authentication	90%	80%	5
Lab Management	88%	78%	6
Equipment Management	87%	75%	7
Scheduling System	91%	82%	7
Attendance Management	89%	80%	5

Experiment 9

OBJECTIVE: Design test suite for equivalence class partitioning.

1. Equivalence Classes

Class Type	Equivalence Class Description	Example Value
Valid	Length between 3 and 50 characters	"Computer Lab"
Invalid	Length less than 3 characters	"AI"
Invalid	Length more than 50 characters	"Advanced Computer Programming Laboratory Block A Room 101"
Invalid	Empty or null value	"" or null
Invalid	Non-string input	12345

2. Test Cases

Test Case ID	Input	Expected Result	Class
TC01	"Computer Lab"	Accepted	Valid
TC02	"AI"	Rejected – too short	Invalid
TC03	51-character string	Rejected – too long	Invalid
TC04	"" (empty string)	Rejected – required field	Invalid
TC05	null	Rejected –	Invalid

		required field	
TC06	12345	Rejected – invalid data type	Invalid

3. Conclusion

By applying Equivalence Class Partitioning, the input domain is divided into valid and invalid groups. This reduces the number of test cases while ensuring effective coverage of all possible input scenarios.

Experiment 10

OBJECTIVE : Design test cases for Boundary value analysis

Context from SRS (IEEE Format)

From the SRS, consider the following requirements:

- Lab Name must be between 3 to 50 characters
 - Equipment Quantity must be between 1 to 1000 units
- These inputs are critical and tested using Boundary Value Analysis.

1. Boundary Analysis for Lab Name Length (3–50 characters)

Test Case ID	Input (Length)	Expected Result	Remarks
TC01	2	Rejected – below minimum	Invalid boundary
TC02	3	Accepted – at lower limit	Valid boundary
TC03	4	Accepted – just above lower limit	Valid
TC04	49	Accepted – just below upper limit	Valid
TC05	50	Accepted – at upper limit	Valid boundary
TC06	51	Rejected – above maximum	Invalid boundary

2. Boundary Analysis for Equipment Quantity (1–1000)

Test Case ID	Input (Quantity)	Expected Result	Remarks
TC07	0	Rejected – below minimum	Invalid boundary
TC08	1	Accepted – at lower limit	Valid boundary
TC09	2	Accepted – just above lower limit	Valid
TC10	999	Accepted – just below upper limit	Valid
TC11	1000	Accepted – at upper limit	Valid boundary
TC12	1001	Rejected – above maximum	Invalid boundary

3. Conclusion

Boundary Value Analysis focuses on testing values at the edges of input ranges. By testing minimum, maximum, and near-boundary values, we can identify errors that commonly occur at limits.

In the Lab Management System, applying BVA ensures:

- Accurate validation of lab names and equipment data
- Prevention of invalid inputs
- Improved system reliability and robustness

